

Multilevel Queue and Feedback Scheduling

Multilevel Queue (MLQ) and Multilevel Feedback Queue (MLFQ)

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

By the end of this topic, students should be able to:

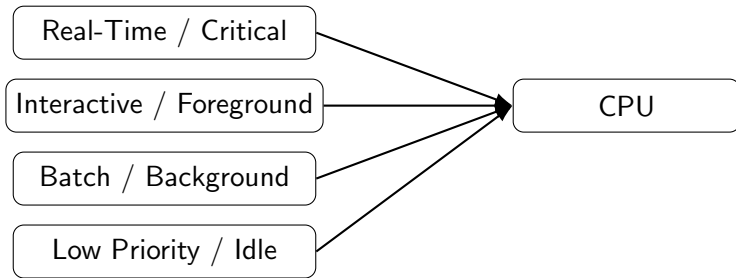
- Explain the motivation for multilevel scheduling.
- Describe the structure of Multilevel Queue (MLQ) scheduling.
- Explain how processes are assigned to different queues.
- Analyze fixed-priority and time-sliced scheduling between queues.
- Explain Multilevel Feedback Queue (MLFQ) scheduling.
- Distinguish MLQ from MLFQ.
- Construct Gantt charts for MLQ and MLFQ examples.
- Calculate CT, TAT, WT and RT for scheduling examples.
- Analyze starvation, aging, responsiveness and scheduling overhead.

Why Do We Need Multilevel Scheduling?

A computer system may execute very different types of processes:

- Interactive foreground applications
- Background batch jobs
- System processes
- Real-time or high-priority tasks
- CPU-intensive computations

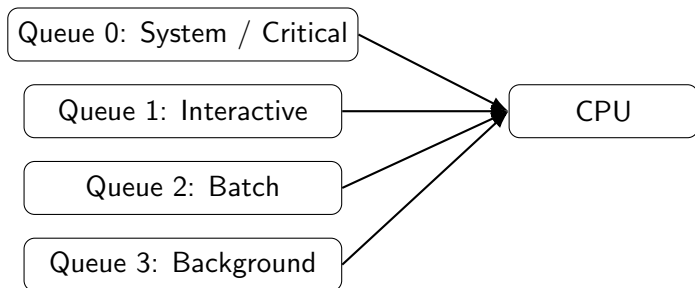
A single scheduling policy may not provide the desired behavior for all of them.



What Is Multilevel Queue Scheduling?

Multilevel Queue (MLQ) divides the ready queue into multiple separate queues.

- Each queue represents a process category.
- A process is permanently assigned to one queue.
- Each queue may use a different scheduling algorithm.
- Scheduling also needs a policy for selecting among queues.



Typical MLQ Queue Organization

One possible organization is:

Queue	Process Type	Scheduling Policy
Q ₀	System processes	Priority / RR
Q ₁	Interactive processes	Round Robin
Q ₂	Batch processes	FCFS
Q ₃	Background jobs	FCFS / low-priority RR

Important

The exact number of queues, process categories and scheduling policies are design choices.

Permanent assignment

In conventional MLQ, a process does not move between queues during its lifetime.

Two-Level MLQ Example

Suppose the system has:

- Queue 1: Interactive processes
- Queue 2: Batch processes

Queue policies:

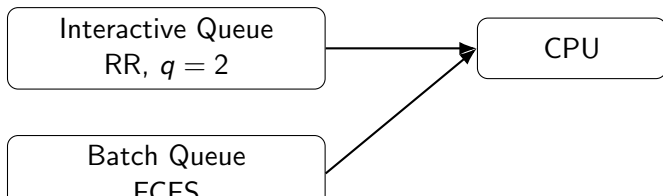
Q_1 : *Round Robin*, $q = 2$

Q_2 : *FCFS*

Between queues:

$Q_1 > Q_2$

This means the interactive queue receives CPU service whenever it contains a ready process.



How Are Queues Selected in MLQ?

Two common approaches are:

1. Fixed Priority Between Queues

Always serve the highest-priority non-empty queue.

Example:

$$Q_0 > Q_1 > Q_2$$

If Q_0 contains a ready process, lower queues do not receive CPU service.

Risk: starvation of lower queues.

2. Time Slice Between Queues

Give each queue a percentage of CPU time.

Example:

$$Q_0 = 50\%, \quad Q_1 = 30\%, \quad Q_2 = 20\%$$

This can provide more predictable CPU sharing.

MLQ: Fixed-Priority Scheduling Example

Assume:

$$Q_0 > Q_1 \quad Q_0 : P_1(AT = 0, BT = 4)$$

$$Q_1 : P_2(AT = 0, BT = 3), \quad P_3(AT = 0, BT = 2)$$

Fixed priority:

$$Q_0 > Q_1$$

Therefore:



P_2 and P_3 cannot execute while the higher-priority Q_0 contains P_1 .

MLQ: Numerical Performance Calculation

For the previous example:

Process	AT	BT	CT	Queue
P ₁	0	4	4	Q ₀
P ₂	0	3	7	Q ₁
P ₃	0	2	9	Q ₁

$$TAT = CT - AT$$

$$WT = TAT - BT$$

P	CT	TAT	WT	RT
P ₁	4	4	0	0
P ₂	7	7	4	4
P ₃	9	9	7	7

$$Avg. WT = \frac{0 + 4 + 7}{3} = \boxed{3.67}$$

MLQ with Time Sharing Between Queues

Suppose:

$$Q_0 = 80\%, \quad Q_1 = 20\%$$

The scheduler reserves CPU time for both queues.

$Q_0: 80\%$	$Q_1: 20\%$
-------------	-------------

Advantage

A lower-priority queue can receive CPU service even when the higher-priority queue remains busy.

Design consideration

The percentages must reflect system objectives and workload requirements.

Advantages and Limitations of MLQ

Advantages

- Simple conceptual structure
- Different policies for different classes
- Can prioritize important workloads
- Predictable queue membership
- Useful for workload isolation

Limitations

- Fixed assignment can be inflexible
- Lower queues may starve
- Classification must be decided in advance
- Workload behavior may change
- Queue priorities require careful tuning

Why Do We Need Feedback Scheduling?

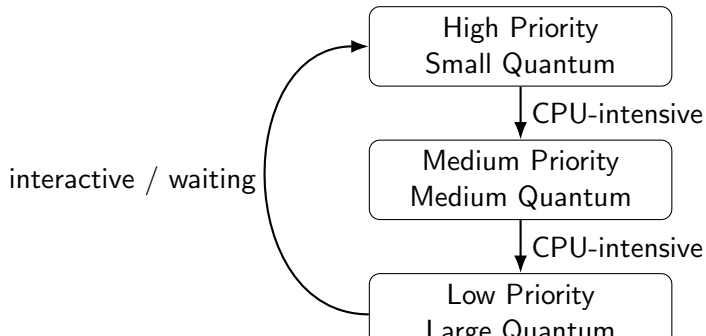
Consider a process initially believed to be interactive. Later, it performs a long CPU-intensive computation.

With fixed MLQ:

Process remains in the original queue

With feedback scheduling:

Process can move between queues



What Is Multilevel Feedback Queue?

Multilevel Feedback Queue (MLFQ) extends MLQ by allowing processes to move between queues.

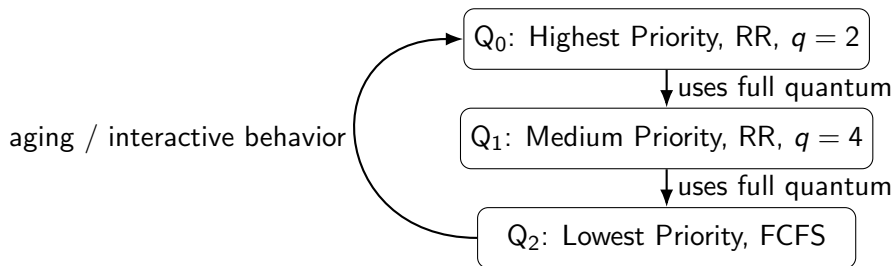
- Processes can move upward or downward.
- CPU-intensive processes may be moved to lower-priority queues.
- Interactive or I/O-bound processes may remain at higher priority.
- Aging can promote long-waiting processes.
- Queue scheduling policies can differ.

Core idea

The scheduler uses observed process behavior to dynamically adjust its scheduling priority.

Typical MLFQ Structure

A common conceptual design:



Interpretation

Short or interactive jobs tend to receive quick service, while long CPU-bound jobs gradually move toward lower queues.

Typical MLFQ Rules

A simplified MLFQ can use these rules:

- 1 Always execute the highest-priority non-empty queue.
- 2 Within Q_0 , use Round Robin with a small quantum.
- 3 If a process uses its entire quantum, move it down.
- 4 If a process frequently blocks for I/O, keep it at a higher level.
- 5 Periodically boost waiting processes to prevent starvation.

Priority is dynamically adjusted from observed behavior

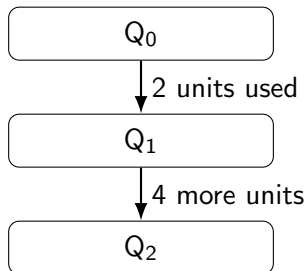
Important

There is no single universal MLFQ configuration. Real systems can use significantly more sophisticated policies.

MLFQ Example: Process Movement

$Q_0 : q = 2,$ $Q_1 : q = 4,$ $Q_2 : FCFS$

Process P_1 requires 10 CPU units.



Remaining CPU demand: $10 - 2 - 4 = 4$

Thus a CPU-bound process can gradually move to lower queues.

Reason

The scheduler interprets repeated full-quantum usage as evidence of CPU-intensive behavior.

MLFQ Example: Interactive Process

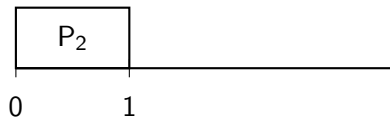
Suppose P_2 needs:

$$CPU\ burst = 1$$

and arrives in Q_0 with:

$$q = 2$$

It completes before using the full quantum.



P_2 finishes quickly.

Interpretation

Short jobs benefit from remaining in a high-priority queue with a small quantum.

MLFQ Numerical Example

Assume:

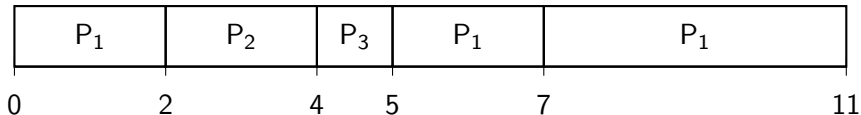
Process	Arrival Time	CPU Burst
P ₁	0	8
P ₂	1	2
P ₃	2	1

MLFQ: Q₀: RR, $q = 2$, Q₁: FCFS

Simplified rule:

- New processes enter Q₀.
- A process using its entire quantum moves to Q₁.
- Q₀ has higher priority than Q₁.

A possible schedule is:



MLFQ: Performance Calculation

P	AT	BT	ST	CT	TAT	WT
P ₁	0	8	0	11	11	3
P ₂	1	2	2	4	3	1
P ₃	2	1	4	5	3	2

Response time:

$$RT_{P_1} = 0 - 0 = 0$$

$$RT_{P_2} = 2 - 1 = 1$$

$$RT_{P_3} = 4 - 2 = 2$$

Average values:

$$Avg.WT = \frac{3 + 1 + 2}{3} = \boxed{2}$$

MLQ vs. MLFQ

Feature	MLQ	MLFQ
Queue assignment	Usually fixed	Dynamic
Process movement	No	Yes
Priority adaptation	Limited	Central feature
Scheduling policy	Different per queue	Different per queue
Workload adaptation	Low	High
Starvation handling	Requires queue policy / aging	Aging and periodic boosts often used
Complexity	Lower	Higher
Main idea	Separate classes	Adapt to behavior

MLQ = fixed classification

MLFQ = dynamic classification + feedback

Starvation in MLQ and MLFQ

MLQ problem:

If:

$$Q_0 > Q_1$$

and Q_0 is continuously busy, Q_1 may receive little or no CPU time.

MLFQ problem:

A process moved to a low-priority queue can also wait for a long time.

Common solutions

- Aging
- Periodic priority boost
- Time-sharing between queues
- Maximum waiting-time constraints

Fairness requires controlling indefinite waiting

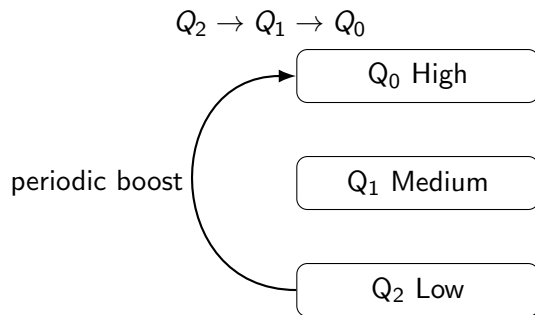
Priority Boost in MLFQ

Suppose a process has remained in Q_2 for a long time.

Without a boost:

$$Q_2 \rightarrow Q_2 \rightarrow Q_2 \rightarrow \dots$$

With periodic priority boost:



Real-World Analogy

MLQ analogy: Different service counters

A bank may have separate counters for:

- Priority customers
- General customers
- Business customers

A customer remains assigned to a category.

MLFQ analogy: Adaptive service lanes

A service system may initially serve everyone quickly, but repeatedly large requests are moved to a slower lane while small requests remain in the fast lane.

OS interpretation

The scheduler uses process behavior to determine how aggressively CPU service should be allocated.

Technical Example: Interactive vs. CPU-Bound Process

Consider:

- P_1 : text editor, frequently waits for keyboard input.
- P_2 : video encoding, continuously consumes CPU.

In MLFQ:

$$P_1 \rightarrow Q_0$$

because it frequently blocks before exhausting its quantum.

$$P_2 \rightarrow Q_1 \rightarrow Q_2$$

because it repeatedly consumes full quanta.

Result

The interactive process receives quick CPU service while the CPU-bound job still receives substantial CPU time.

Advantages of MLFQ

- Adapts to changing process behavior.
- Good response for interactive workloads.
- Can favor short CPU bursts without knowing exact burst lengths.
- Separates CPU-bound and interactive behavior dynamically.
- Aging / priority boosts can provide fairness.
- Can combine multiple scheduling algorithms.

Main cost

MLFQ is considerably more complex than FCFS or basic Round Robin.

Limitations and Design Challenges of MLFQ

- Choosing the number of queues.
- Selecting time quantum for each queue.
- Defining demotion rules.
- Defining promotion rules.
- Preventing starvation.
- Handling priority boosts.
- Controlling context-switch overhead.
- Avoiding pathological workload behavior.

Design principle

The scheduler must balance:

Responsiveness + Fairness + Throughput + Overhead

MLQ vs. Traditional Scheduling Algorithms

Algorithm	Main Selection Rule	Adaptation	Typical Concern
FCFS	Arrival order	None	Convoy effect
SJF	Shortest burst	None	Burst prediction / starvation
SRTF	Shortest remaining time	Arrival-driven	Preemption overhead
RR	Circular queue + quantum	Limited	Quantum selection
Priority	Priority value	Depends on implementation	Starvation / inversion
MLQ	Queue class + queue policy	Usually none	Fixed classification
MLFQ	Queue + observed behavior	High	Complexity / tuning

How to Solve an MLQ/MLFQ Problem

- 1 Identify all queues.
- 2 Identify the scheduling algorithm inside each queue.
- 3 Identify the priority/order between queues.
- 4 Determine whether queues can preempt one another.
- 5 For MLFQ, write the promotion/demotion rules.
- 6 Track process movement between queues.
- 7 Draw the complete Gantt chart.
- 8 Determine first start time and completion time.
- 9 Calculate:

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = ST - AT$$

- 10 Calculate averages and interpret the result.

Common Mistakes

- Confusing MLQ with MLFQ.
- Assuming processes automatically move between MLQ queues.
- Forgetting the scheduling policy inside each queue.
- Forgetting the policy used between queues.
- Ignoring preemption between higher and lower queues.
- Moving a process between MLFQ queues without applying the stated rule.
- Forgetting aging or priority boosts when specified.
- Calculating waiting time directly from the Gantt chart incorrectly.

Exam tip

Before drawing the Gantt chart, write the complete queue-selection and process-movement rules.

HOT Question 1: MLQ Design

A university server executes:

- system processes,
- interactive student applications,
- batch data-processing jobs.

Design a three-level MLQ scheduler.

Specify:

- ① Which process type belongs to each queue.
- ② Scheduling algorithm inside each queue.
- ③ Priority between queues.
- ④ How starvation will be prevented.
- ⑤ Why your design is suitable for the workload.

HOT Question 2: MLQ vs. MLFQ

Suppose a process starts as interactive but later becomes CPU-intensive.

- 1 Explain what happens under MLQ.
- 2 Explain what happens under MLFQ.
- 3 Compare their responsiveness.
- 4 Compare fairness.
- 5 Explain why MLFQ requires more scheduling decisions.

Higher-order focus

Analyze how dynamic process behavior changes the suitability of fixed versus adaptive queue assignment.

HOT Question 3: Quantum Selection

An MLFQ system has:

$$Q_0 : q = 2, \quad Q_1 : q = 4, \quad Q_2 : q = 8$$

A CPU-bound process repeatedly uses its entire quantum.

- 1 Trace its movement through the queues.
- 2 Explain why the process is eventually placed in Q_2 .
- 3 Explain what happens to a process that frequently blocks after 1 time unit.
- 4 Discuss how changing q affects response time and overhead.

HOT Question 4: Starvation Analysis

An MLQ scheduler uses:

$$Q_0 > Q_1 > Q_2$$

with strict priority between queues.

Q_0 continuously receives new processes.

- 1 Explain why Q_2 can starve.
- 2 Suggest two mechanisms to prevent starvation.
- 3 Compare aging with time-sharing between queues.
- 4 Discuss the effect of each mechanism on responsiveness and fairness.

Summary

- **MLQ** divides processes into fixed queues.
- Each MLQ queue can use a different scheduling algorithm.
- Processes generally remain in their assigned queue.
- Strict queue priority can cause starvation.
- Time-sharing between queues can improve fairness.
- **MLFQ** allows processes to move between queues.
- CPU-bound processes can be demoted.
- Interactive / I/O-bound processes can retain higher priority.
- Aging or periodic priority boosts can prevent starvation.

MLQ = fixed queues

MLFQ = dynamic queues + feedback